משימה 3 — הגיבוי שפלנקטון שכח

נושאים: זיכרון דינמי, `malloc`, `calloc`, `realloc`, `free`, בעלות על זיכרון, העתקה עמוקה ו־pointer-to-pointer.

הרצת `./operation_b7` מהתיקייה הראשית יוצרת את `mission3/data/mission3_archive.b7`. זהו ארכיון ראיות אישי; אין לערוך אותו.

תדריך מודיעיני

במשימה הקודמת גיליתם שסוכן 67, שאינו מופיע במאגר הסוכנים הרשמי, קיבל גישה למערכות רגישות, ובהן גם תשתית התקשורת הפרטית של מורטי הרשע.

התשדורות שיורטו מצביעות גם על מעורבות של גורמים איראניים בהכנות למתקפה על באר שבע, אך עדיין לא ברור מה היקף המעורבות שלהם ומה בדיוק מתכננים המפעילים.

הבדיקה שביצעתם ביומן הגישה לא נעלמה מעיניו של מורטי. כשזיהה שמישהו עוקב אחר פעילות הארגון, הוא הורה לפלנקטון למחוק את ההתכתבויות.

האירוע הזה מחייב אותנו להיות זהירים יותר בהמשך החקירה. מורטי אמנם אינו יודע מי עומד מאחורי הבדיקות, אך כעת הוא מודע לכך שמישהו מנסה לחשוף את פעילות הארגון.

ההודעות אכן נעלמו מהשרת. גם הגיבוי שפלנקטון הכין לא עזר במיוחד:

> **מורטי הרשע:** פלנקטון, מחקת את כל ההתכתבויות?
>
> **פלנקטון:** בוודאי! השתמשתי ב־`free` בדיוק כמו שלימדו אותי!
>
> **מורטי הרשע:** ומה עם הגיבוי?
>
> **פלנקטון:** ברור שעשיתי גיבוי. העתקתי את המצביע פעמיים.
>
> **מורטי הרשע:** אתה מבין שזה עדיין מצביע לאותו מקום בזיכרון?
>
> **פלנקטון:** לא נכון. שמתי אותם בשני משתנים שונים.
>
> **מורטי הרשע:** ...
>
> **פלנקטון:** אני מתחיל להבין למה השרת קרס.

הגיבוי שפלנקטון הכין בזיכרון היה למעשה אוסף של מצביעים לאותן הודעות. ברגע ששחרר את הזיכרון המקורי, גם העותק שלו הפך לחסר תועלת.

הגיבוי שאותר בסוף החקירה הקודמת הוא **ארכיון ישן שנשמר בנפרד, באחסון קבוע**. מחיקת ההודעות מהזיכרון לא בהכרח מחקה גם את הארכיון הזה. ייתכן שההתכתבויות שמורטי ניסה להעלים עדיין שמורות בו.

הבעיה היא שתוכנת השחזור אינה מסוגלת להחזיק את ההודעות בזיכרון באופן אמין. ללא ניהול תקין של ההקצאות וההעתקות, הודעות עלולות להיפגע או להיעלם עוד לפני שניתן יהיה לבדוק אותן.

המשימה שלכם

עליכם לשחזר את ההתכתבויות שמורטי ניסה למחוק. זהו קצה החוט הישיר ביותר שיש כרגע לחקירה, בזמן שההכנות למתקפה על באר שבע נמשכות. צוות המודיעין הצליח לשמר את הארכיון; האחריות להוציא ממנו ראיות שאפשר לסמוך עליהן מוטלת עליכם.

לשם כך תבנו ב־C את שכבת ניהול הזיכרון של תוכנת השחזור: תקצו מקום להודעות, תרחיבו את המאגר לפי הצורך, תיצרו עותקים עצמאיים ותשחררו זיכרון בלי לפגוע במידע. כל שלב שתשלימו יאפשר לבחון חלק נוסף מההתכתבויות.

הארכיון עצמו כבר מסופק לכם. **אין לערוך את קובץ הראיות או לנסות להחליף את תוצאות השחזור בפלט קבוע.** עליכם לממש את הפונקציות ב־`messages.c`, בהתאם לחוזים שלהלן.

מבני הנתונים וכללי הבעלות

המבנים מוגדרים עבורכם ב־`messages.h`:

```
typedef struct {
    int id;
    char *text;
} Message;

typedef struct {
    Message *messages;
    size_t count;
    size_t capacity;
} MessageStore;
```

`MessageStore` היא **הבעלים** של מערך `messages`, וכל הודעה תקינה במערך היא הבעלים של מחרוזת `text` נפרדת שהוקצתה עבורה.

מחרוזת שמתקבלת כ־`* const char` היא קלט **מושאל**: מותר לקרוא ולהעתיק אותה, אך אסור לשמור תלות בכתובת שלה ואסור לשחרר אותה. המאגר חייב להמשיך לעבוד גם אם מחרוזת המקור השתנתה או כבר אינה קיימת.

מאגר ריק תקין מקיים:

```
messages == NULL
count == 0
capacity == 0
```

כל פונקציה שמחזירה `int` תחזיר `1` בהצלחה ו־`0` בכישלון. מזהה הודעה חייב להיות חיובי; מחרוזת ריקה היא הודעה חוקית. אין להשתמש ב־`strdup` או בהקצאות שאינן חלק מ־C11.

שלב A — קליטת ההודעות הראשונות

כדי להתחיל בשחזור, צריך להקצות מאגר להודעות ולקלוט לתוכו מחרוזות בלי להסתמך על הזיכרון שממנו נקראו.

ממשו ב־`messages.c`:

```c
int store_init(MessageStore *store, size_t initial_capacity);
int message_init(Message *msg, int id, const char *text);
```

`store_init`

- הפונקציה מקבלת מאגר יעד ריק.
- אם הקיבולת היא אפס, אתחלו מאגר ריק ללא הקצאה.
- אחרת הקצו באמצעות `calloc` מערך של `initial_capacity` מבני `Message`, כך שכל התאים יתחילו במצב ריק.
- יש למנוע overflow בחישוב גודל ההקצאה.
- בכישלון השאירו את המאגר במצב הריק התקין.

`message_init`

- הפונקציה מקבלת `Message` ריק, מזהה חיובי ומחרוזת מקור שאינה `NULL`.
- הקצו באמצעות `malloc` מקום לכל התווים ול־`'\0'`, והעתיקו את המחרוזת במלואה.
- קבעו את המזהה רק לאחר שהאתחול הצליח.
- בכישלון השאירו `id == 0` ו־`text == NULL`.

לבדיקה, מתוך `/mission3`:

```
make test-a
```

הבדיקות מכסות גם מאגר בקיבולת אפס, הודעות ריקות, עצמאות מהקלט וכשלי הקצאה. לאחר הצלחה, הריצו את השלב ועיינו בממצאים שמוצגים.

שלב B — הרחבת מאגר ההודעות

הארכיון מכיל יותר הודעות מכפי שאפשר להבטיח שייכנסו למאגר הראשוני. תוכנת השחזור צריכה להגדיל את הזיכרון תוך שמירה על מה שכבר נקלט.

ממשו:

```c
int store_reserve(Message **messages, size_t *capacity, size_t required);
int store_append(MessageStore *store, int id, const char *text);
```

`store_reserve`

- אם הקיבולת מספיקה, החזירו הצלחה בלי לשנות כתובות או נתונים.
- אחרת הגדילו את המערך באמצעות `realloc` ומדיניות גידול גאומטרית סבירה. בקיבולת אפס התחילו מקיבולת קטנה.
- השתמשו ב־`Message **` כדי לעדכן את המצביע של הקורא רק לאחר הצלחה. אם `realloc` נכשל, המצביע והקיבולת המקוריים חייבים להישאר תקינים.
- אתחלו כל תא חדש ל־`id == 0` ו־`text == NULL`.

- מנעו overflow הן בגידול הקיבולת והן בכפל ב־`sizeof(Message)`.

`store_append`

- הגדילו את המאגר בעת הצורך באמצעות `store_reserve`.
- צרו הודעה בבעלות עצמאית באמצעות `message_init`.
- הגדילו את `count` רק לאחר שההודעה נוצרה בהצלחה.
- בכישלון שמרו על כל ההודעות הקיימות. מותר שהקיבולת כבר גדלה לפני שכשל `malloc` של המחרוזת.

לבדיקה:

```
make test-b
```

לאחר הצלחה, עיינו בתוצאות השחזור של שלב זה.

שלב C — ניקוי בטוח של הזיכרון

לפני שיוצרים גיבוי מורכב, צריך לדעת לשחרר בבטחה מאגר חלקי או מלא. כך ניתן יהיה להשתמש באותן פונקציות ניקוי גם אם הקצאה תיכשל בשלב הבא.

ממשו:

```
void message_free(Message *msg);
void store_free(MessageStore *store);
```

- `message_free` משחררת את המחרוזת שבבעלות ההודעה ומאפסת את המצביע והמזהה.
- `store_free` משחררת תחילה את המחרוזות של `count` ההודעות התקינות, לאחר מכן את המערך, ולבסוף מאפסת את כל שדות המאגר.
- `NULL`, הודעה ריקה, מאגר ריק וקריאות ניקוי חוזרות חייבים להיות בטוחים.
- אין לשחרר מחרוזות מושאלות, לשחרר אותה הקצאה פעמיים או להשתמש בזיכרון לאחר `free`.

לבדיקה:

```
make test-c
```

הבדיקות מוודאות שהזיכרון משתחרר בסדר הנכון, שהמצב מתאפס ושקריאות חוזרות בטוחות.

שלב D — יצירת גיבוי שאפשר לסמוך עליו

כדי שהראיות לא יהיו תלויות בעותק יחיד בזיכרון, צוות החקירה צריך ליצור מהמאגר המשוחזר עותק עצמאי לחלוטין. העתקת הכתובות בלבד לא תספיק — כפי שפלנקטון כבר הדגים.

ממשו:

```
int store_clone(MessageStore *dest, const MessageStore *source);
```

- שני הפרמטרים מתארים מאגרים תקינים, ו־`dest` ריק בתחילת הקריאה.
- צרו **העתקה עמוקה**: מערך חדש ומחרוזת חדשה לכל אחת מ־`count` ההודעות.
- שמרו על הסדר, המזהים והטקסט. הקיבולת של העותק יכולה להיות שונה מקיבולת המקור, כל עוד `dest->capacity >= dest->count`.
- מקור ריק משוכפל בהצלחה למאגר ריק.
- אם הקצאה כלשהי נכשלת באמצע, השתמשו בפונקציות הניקוי משלב C כדי לשחרר את מה שכבר הוקצה ולהשאיר את `dest` ריק.
- אין לשנות את המקור. העותק חייב להישאר תקין לאחר שינוי או שחרור המקור.

לבדיקה:

```
make test-d
```

הבדיקות מוודאות שהעותק אינו חולק את מערך ההודעות או את המחרוזות עם המקור, וגם שכשל באמצע ההעתקה אינו משאיר זיכרון מוקצה או נתונים פגומים.

הרצת החקירה המלאה

לאחר שכל השלבים עוברים, הריצו מתוך `/mission3`:

```
make run
```

תשתית הקורס טוענת את הארכיון ומוסרת לפונקציות שלכם מחרוזות מושאלות. במהלך הריצה נתוני הטעינה משוחררים, והמערכת בודקת שההודעות והעותק העמוק נשארו תקינים ועצמאיים. אם כל השלבים הושלמו בהצלחה, יופיעו ממצאי השחזור וקוד ההשלמה האישי בפורמט `B7-M3-XXXXXXXX`.

אל תסתפקו בקוד ההשלמה. קראו את ההתכתבויות ששוחזרו, וכתבו בדוח מה ניתן ללמוד מהן על פעילות הארגון.

בנייה וקובצי התשתית

`make` מקמפל את `messages.c` ומקשר אותו עם תשתיות הקורס. קובץ המקור שלכם מספק את פונקציות ניהול הזיכרון; התשתית מספקת את טעינת הארכיון, הבדיקות והצגת הראיות, אך אינה ממשת אף פונקציה מה־header המקומי `messages.h`.

אפשר למחוק את קובץ ההרצה באמצעות `make clean`. אין למחוק את הספרייה או את הארכיון שנוצר.

דוח החקירה והגשה

מלאו את חלק משימה 3 ב־`investigation_report.md`, בהתאם לשאלות המופיעות בו:

- **ממצאים:** מה למדתם מההתכתבויות ששוחזרו, וכיצד המידע משפיע על המשך המבצע? ציינו את קוד ההשלמה.

- **בעלות על זיכרון:** הסבירו את ההבדל בין קלט מושאל לבין זיכרון שבבעלות `MessageStore`. מה מותר לשחרר ומי אחראי לכך?
- **טיפול בכשל הקצאה:** מה צריך לקרות אם הקצאה נכשלת באמצע `store_clone`? הסבירו מה קורה להקצאות שכבר בוצעו ולמצב של `dest`.

ענו בקצרה ובמילים שלכם, תוך התייחסות למימוש בפועל. אין צורך לתאר כל פונקציה או לחזור על כל שלבי המימוש.

הגישו:

- `mission3/messages.c`
- `investigation_report.md` המעודכן